

Relatório: Semana 03 – Execução e Simulação das Regras

Disciplina: Compiladores

Curso: Ciência da Computação

Docente: Prof.^a Ma. Layse Souza

Grupo: 05

1. Objetivo

O objetivo principal desta etapa foi implementar o módulo de **Análise Semântica Avançada** e **Validação de Regras de Negócio** para a linguagem RoboLang.

Diferente da análise sintática (que verifica a estrutura gramatical), esta fase foca em garantir que as instruções façam sentido lógico dentro do contexto de operação do robô. Os objetivos específicos incluíram:

- Converter a árvore de análise bruta em uma Árvore Sintática Abstrata (AST) manipulável.
- Aplicar restrições de valores (faixas aceitáveis para ângulos, distâncias e tempo).
- Validar unidades de medida e coerência dos parâmetros.
- Implementar um sistema robusto de coleta e reporte de erros semânticos.

2. Estrutura Implementada

A arquitetura da solução foi expandida para suportar a verificação profunda dos dados. O sistema opera em pipeline:

1. **Construtor de AST (ASTBuilder):** Responsável por visitar a árvore gerada pelo analisador sintático e transformá-la em objetos Python estruturados, facilitando a manipulação posterior e a exportação para JSON.
2. **Analisador Semântico (SemanticAnalyzer):** É o núcleo desta etapa. Esta classe recebe a AST e percorre cada nó (comandos e declarações) aplicando regras de validação específicas. Ela possui um mecanismo interno para acumular múltiplos erros, permitindo que o programador veja todas as falhas de uma só vez, em vez de parar na primeira ocorrência.
3. **Classes de Erro Especializadas:** Foram implementadas classes de exceção específicas para categorizar os problemas, como `InvalidValueError` (valor inválido), `InvalidUnitError` (unidade desconhecida) e `InvalidAngleError` (ângulo fora do limite).

3. Regras de Validação e Verificações

O analisador semântico foi configurado para barrar comandos que, embora sintaticamente corretos, são logicamente inválidos para o robô. As principais regras implementadas foram:

- **Movimentação (Mover):**
 - A distância deve ser estritamente positiva (valores negativos ou zero geram erro).
 - A unidade de medida deve ser válida ('cm' ou 'm').
 - O eixo de movimentação deve pertencer ao espaço cartesiano suportado ('x', 'y' ou 'z').
- **Rotação (Virar):**
 - O ângulo deve estar compreendido no intervalo entre 0 e 360 graus. Valores como 400° ou negativos são rejeitados.
- **Estados e Tempo (Velocidade e Esperar):**
 - A velocidade configurada deve ser positiva.
 - O tempo de espera (delay) deve ser um valor positivo.
- **Controle de Fluxo (Repetir):**
 - A contagem de repetições deve ser um número inteiro maior ou igual a 1. Loops com contagem 0 são considerados inválidos.
- **Integridade do Programa:**
 - O programa deve possuir um identificador (nome) válido e não vazio.

4. Testes e Resultados

Para garantir a robustez do analisador, foi desenvolvida uma suíte de testes automatizada que processa arquivos `.robo` e classifica-os como VÁLIDOS ou INVÁLIDOS.

Cenários de Teste Executados:

1. **Programas Válidos:**
 - Exemplos contendo instruções corretas de movimentação linear, curvas de 180° e loops de repetição foram processados com sucesso, recebendo o status "✅ SEMANTICAMENTE VÁLIDO".
2. **Programas Inválidos (Detecção de Erros):**
 - **Erro de Magnitude:** Tentativa de mover x -10 cm. O sistema detectou e reportou que a distância deve ser positiva.
 - **Erro de Limite:** Tentativa de virar 400 graus. O sistema identificou que o valor excede o limite físico de 360°.
 - **Erro de Lógica:** Tentativa de repetir 0. O sistema alertou que a contagem deve ser pelo menos 1.
 - **Múltiplos Erros:** Em um cenário complexo, o analisador foi capaz de listar simultaneamente erros de velocidade zero, espera nula e ângulos incorretos, demonstrando a capacidade de coleta de erros múltiplos.

5. Conclusão

A implementação da Semana 03 atingiu seus objetivos ao garantir que apenas programas logicamente consistentes sejam processados. O sistema agora é capaz de:

- Validar regras de negócio complexas que a gramática sozinha não consegue capturar.
- Fornecer feedback detalhado e semântico para o desenvolvedor (indicando exatamente qual regra foi violada).
- Gerar uma AST validada e segura.

Com a validação semântica concluída, o compilador RoboLang está pronto para a próxima fase de geração de código final, com a garantia de que os dados de entrada respeitam as restrições físicas e lógicas do robô.